



调用存储过程

北京理工大学计算机学院
金旭亮

存储过程



存储过程（Stored Procedure）是一组可以完成特定功能的SQL语句集合，经编译后存储在数据库中。



存储过程具有非常独特的优点，比如，它的执行速度快，表达力强，可以很方便地在数据库层直接实现许多业务逻辑，而不再需要使用Java/C#等编程语言编写实现相应的业务逻辑层，从而大大地提升了开发效率。



历史上存储过程用得非常多，不过随着信息化系统的“Web化浪潮”，存储过程的使用需求在下降，它所承担的功能，人们越来越多地趋向于使用中间层组件实现，这个主要是为了应付分布式系统的挑战，保证系统与代码具有比较好的可扩展和伸缩性。

不同的数据库，使用不同的语言创建存储过程

你可以把存储过程想象为由“数据库管理系统（DBMS）”执行的函数，只不过这个“函数”本身，是使用各数据库支持的“扩展了的”SQL命令编写的。



Stored Procedure Language

ORACLE®

PL/SQL
PL/SQL stands for
Procedural language extensions to
the Structured Query Language (SQL)

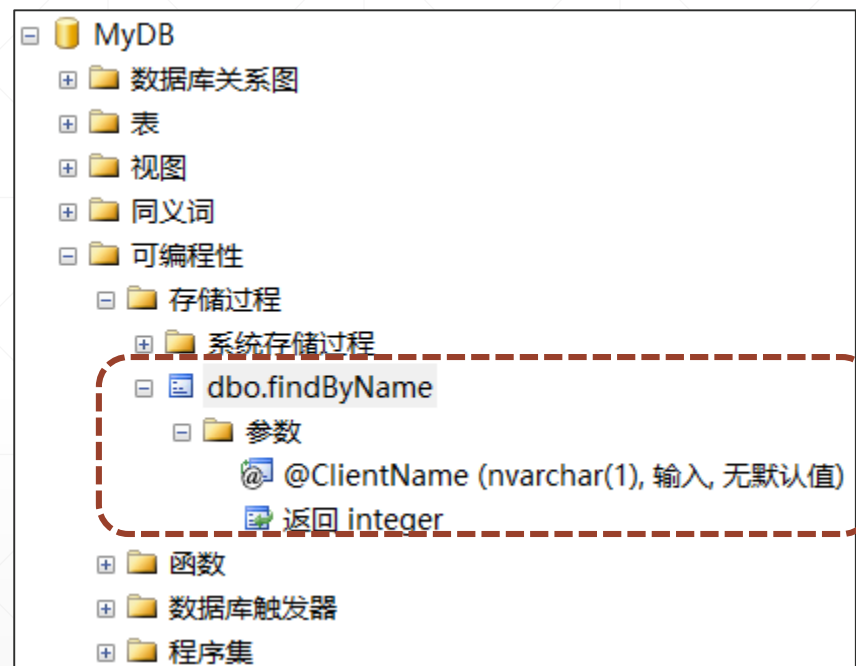


Transact SQL or TSQL

一些大型数据库，比如SQL Server，还可以使用通用编程语言（比如C#）编写“存储过程”，其实就是数据库服务器内置了一个编程语言（比如C#）轻量级运行环境，可以直接跑相应语言编写的应用程序（比如.NET程序）。

创建示例存储过程 (SQL Server)

```
CREATE PROCEDURE findByName
    @ClientName nvarchar
AS
BEGIN
    SET NOCOUNT ON;
    SELECT * from OrderClient where OrderClient.ClientName
        Like '%' + @ClientName + '%'
END
```



存储过程的调用方法



我们可以使用JDBC的CallableStatement执行存储过程，需要指定存储过程的名字并给出参数值（如果该存储过程带有参数）来执行它，通常会得到三种返回值类型：



存储过程返回一个整数值



存储过程使用output类型参数向调用者返回单个结果



存储过程返回一个记录集，JDBC将得到一个ResultSet

JDBC使用CallableStatement执行存储过程

1

使用Connection对象的prepareCall方法创建CallableStatement对象：

```
CallableStatement cs = conn.prepareCall("{call 过程名(参数表)}" );
```

2

依据参数类型，调用setXXX()方法设置相关的参数。

```
cs.setString(参数位置索引, “参数值” );
```

3

依据存储过程的返回值类型，调用以下方法（之一）执行存储过程并接收结果

- cs.execute();
- cs.executeQuery();
- cs.executeUpdate() ;

使用CallableStatement调用存储过程实例

//调用存储过程

```
private static void callStoreProcedure() throws SQLException {
    try (var connection :Connection =
        DriverManager.getConnection(MsSqlServer,
            userName, password)){

        //尝试调用SQL Server数据库中的findByName这一存储过程
        String sql = "{call findByName(?)}";
        CallableStatement callableStatement =
            connection.prepareCall(sql, ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY);

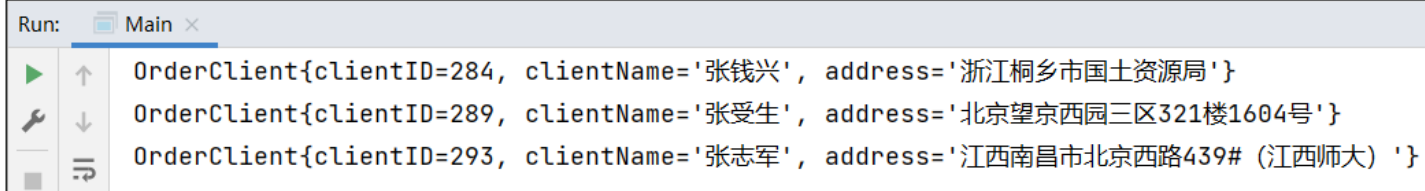
        //设置存储过程需要的输入型参数
        callableStatement.setString( parameterIndex: 1, x: "张");

        //执行查询, 得到结果集
        ResultSet rs = callableStatement.executeQuery();

        //将结果集转换为Java对象集合
        var clients :List<OrderClient> = getClients(rs);

        //显示对象集合中的数据
        clients.forEach(System.out::println);
    }
}
```

运行截图：



```
Run: Main x
OrderClient{clientID=284, clientName='张钱兴', address='浙江桐乡市国土资源局'}
OrderClient{clientID=289, clientName='张受生', address='北京望京西园三区321楼1604号'}
OrderClient{clientID=293, clientName='张志军', address='江西南昌市北京西路439# (江西师大) '}
```

多学一点：处理输出型参数



有些存储过程包容输出型参数，这时，可以用CallableStatement的registerOutParameter()方法声明一个输出型变量，之后再调用getXXX()方法读取其值，以下是示例代码：

```
Connection conn = ...;

//第一个参数是输入型参数，第二个参数是名为outputArgu的输出型参数，是一个整数类型的参数
String SQL = "{call 存储过程名 (?, ?)}";

CallableStatement stmt = conn.prepareCall(SQL,
        ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
) {
    //设置输入型参数值，这里设置为100
    stmt.setDouble(1, 100);
    //注册输出型参数
    stmt.registerOutParameter("outputArgu", Types.INTEGER);
    //执行存储过程，存储过程执行完毕后，输出型参数应该有值了
    rs = stmt.executeQuery();
    int result = stmt.getInt("outputArgu"); //读取输出型参数的值
    //....
}
```


小结



本讲介绍了JDBC调用数据库存储过程的基本方法。



早些年，在行业管理软件中，由于企业有专门经费购买Oracle等大型商业数据库，因此，存储过程用得非常多。



进入互联网时代以后，NoSQL数据库（比如Redis、MongoDB等）兴起，仅用于关系型数据库的存储过程就用得比较少了。



现在推荐的作法，是采用中间件的方式，使用中间层组件实现存储过程的职责，不再推荐编写存储过程，因此，对这块内容，了解即可。